Transformers and Model Families

Self-Attention, the Transformer Block, Encoders and Decoders, and Fine-Tuning

Naeemullah Khan
naeemullah.khan@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

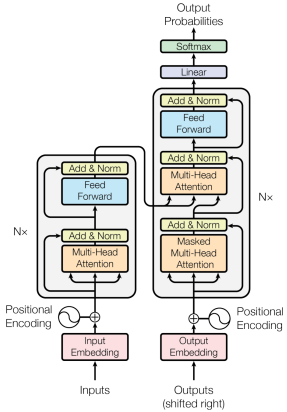
November 9, 2026

Table of Contents

1. Motivation: attention only, no recurrence; encoder, decoder, or both
2. Self-attention: queries, keys and values, worked by hand, in PyTorch, and why scores are scaled
3. Multi-head attention: several heads at the cost of one, the jobs real heads do, and the cost of long inputs
4. Positional encoding: why attention needs it, sinusoidal and learned encodings
5. The transformer block: residual connections, layer normalization, the feed-forward network, parameter counts
6. Causal masking and the decoder: the mask by hand and in code, cross-attention, parallel training
7. Why transformers won: parallel training, short paths, and better translation for less training
8. Pretraining the three families: masked words (BERT, AraBERT), next token (GPT, Qwen), corrupted spans (T5, mBART), and which family for which job
9. Fine-tuning in practice: the Trainer on IMDb, long reviews, pretrained versus from scratch, every model compared
10. Summary and references

Attention Only, No Recurrence

- **The problem:** with attention, the decoder no longer depends on one final vector, but the encoder still builds its states one word after another. A 1,000-word text is 1,000 steps that cannot run in parallel, and the gradient between distant words still crosses every step.
- **The fix:** Vaswani et al. (2017) proposed "the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely".
- Every word attends to every word in one step, so a whole sentence is processed in parallel during training, and any two words are one layer apart.
- English to French: a new single-model record of 41.8 BLEU (n-gram overlap with reference translations, 0 to 100). The figure is the whole model: an encoder on the left, a decoder on the right.



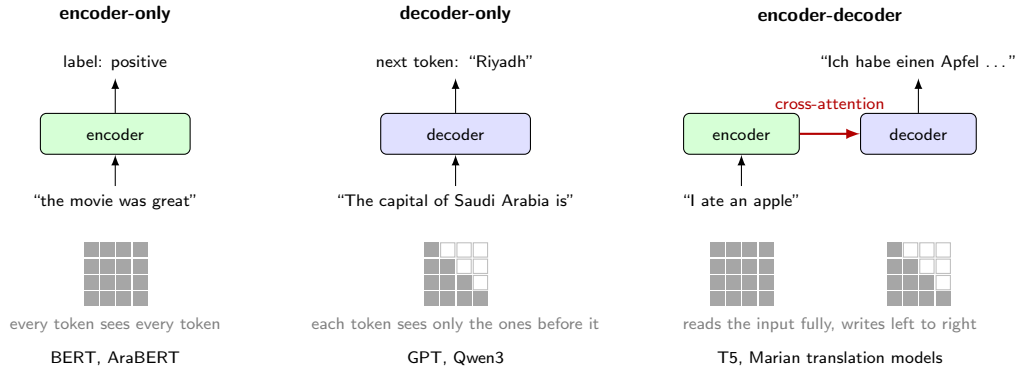
Vaswani et al., "Attention Is All You Need," NeurIPS 2017, arXiv:1706.03762v7, Figure 1.

From the Recurrent Encoder-Decoder to the Transformer

job	recurrent encoder-decoder with attention	transformer
build a vector per word	RNN steps, one after another	self-attention over all words at once
know the word order	built into the recurrence	position information added to the input
decoder reads the source	attention over encoder states	cross-attention, the same idea
decoder reads its own past	recurrent state	self-attention with a causal mask
depth	1–4 recurrent layers	6 to dozens of identical blocks

- Nothing in the right column is new by itself: queries, keys and values, cross-attention and teacher forcing carry over unchanged.
- What is new is removing the recurrence, and the three pieces that make that work: multi-head self-attention, positional encodings, and a block that can be stacked deep.
- The payoff on movie-review sentiment: a small pretrained transformer, fine-tuned, classifies 91.3% of the 25,000 IMDb test reviews correctly, against 84.5% for the best recurrent classifier.

Encoder, Decoder, or Both



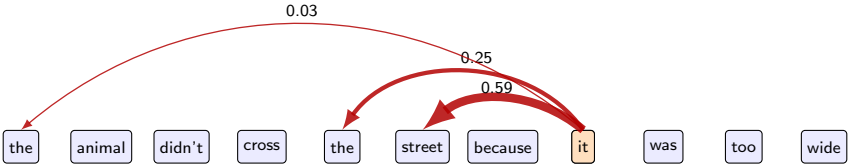
- **Three jobs:** give a text a label, write new text, turn one text into another. Each needs a different half of the transformer: the **encoder** reads, the **decoder** writes, and translation uses both, with the decoder reading the source through cross-attention.
- The blocks are built the same way in all three; what differs is which half is kept and what each token may look at (grey squares: row = token, column = what it may see).

Learning Outcomes

By the end of this lecture you should be able to:

- **Compute** self-attention by hand and in PyTorch, and **explain** why scores are divided by $\sqrt{d_k}$ and why several heads are used.
- **Explain** why a transformer needs position information, and how sinusoidal and learned encodings supply it.
- **Draw** a transformer block and **count** where its parameters are.
- **Apply** a causal mask, and **explain** how it lets a decoder train on every position at once.
- **Explain** why transformers replaced recurrent networks.
- **Distinguish** encoder, decoder and encoder-decoder models by what they see and how they are pretrained, and **choose** one.
- **Fine-tune** a pretrained model with the Hugging Face Trainer, and **decide** between fine-tuning and training from scratch.
- **Prerequisite knowledge:** queries, keys and values; self- and cross-attention; encoder-decoder models, teacher forcing, BLEU; recurrent networks; subword tokenization; residual connections; PyTorch.

Self-Attention: Every Word Asks Every Word



Weights from “it” measured in the trained English encoder `bert-base-uncased`; weights below 0.02 omitted. Computed for this course.

- To represent “it”, the model must work out what “it” refers to: here “street”, because it is the street that is wide. The deciding word comes **after** “it”.
- **The problem:** a left-to-right recurrent network builds the vector of “it” before reading “wide”, from a state passed through every earlier step.
- **The fix, self-attention:** “it” sends a query to every word of its own sentence, before and after it, and its new vector is the weighted mix of their values.
- Every other word does the same at the same time, so one layer rewrites the whole sentence.

Queries, Keys and Values From One Sequence



Source: CMU 11-785, Lecture 19 "Transformers," Spring 2024, slide 68.

- Stack the T input vectors as rows of X ($T \times d_{\text{model}}$). Three learned matrices give

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

with $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

- Then the same formula as for any attention:

$$Z = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- $QK^\top$ is $T \times T$: one score for every pair of positions in the sentence. The softmax runs along each row, so each word's weights sum to 1, and row t of Z is the new vector of word t.

Self-Attention by Hand, 1: Queries, Keys, Values

- Three tokens with two-number embeddings, $d_{\text{model}} = d_k = 2$, as the rows of X:

$$X = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \begin{matrix} \text{the} \\ \text{cat} \\ \text{sat} \end{matrix}$$

- Three weight matrices, the same for every word:

$$W_Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad W_K = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$$

$$W_V = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

word	x	$q = xW_Q$	$k = xW_K$	$v = xW_V$
the	$(1,0)$	$(1,0)$	$(1,1)$	$(1,0)$
cat	$(0,1)$	$(1,1)$	$(0,1)$	$(0,2)$
sat	$(1,1)$	$(2,1)$	$(1,2)$	$(1,2)$
job	input	looks for	offers	hands over

- One row worked: $q_{\text{sat}} = (1,1)\, W_Q = (1+1,\ 0+1) = (2,1)$.
- Each word's one embedding becomes three vectors with three jobs. Stacked as rows: $Q = XW_Q$, $K = XW_K$, $V = XW_V$.
- In a trained model the three matrices are learned; here they are picked by hand.

Self-Attention by Hand, 2: Scores, Weights, Mix

"sat" looks at	1. score $q_{\text{sat}} \cdot k$	scaled, $/\sqrt{2}$	2. softmax weight	3. weight $\times$ value
the	$(2,1)\cdot(1,1) = 3$	2.12	0.31	$0.31\,(1,0)$
cat	$(2,1)\cdot(0,1) = 1$	0.71	0.07	$0.07\,(0,2)$
sat	$(2,1)\cdot(1,2) = 4$	2.83	0.62	$0.62\,(1,2)$
			sum 1.00	sum (0.93, 1.39)

- The sum is the new vector of "sat": 62% from itself, 31% from "the", 7% from "cat".
- "the" and "cat" do the same with their own queries, at the same time. In matrix form: $A = \text{softmax}(QK^\top/\sqrt{2})$, one row per word, and $Z = AV$.

$$A = \begin{pmatrix} 0.40 & 0.20 & 0.40 \\ 0.28 & 0.14 & 0.58 \\ 0.31 & 0.07 & 0.62 \end{pmatrix}, \quad Z = AV = \begin{pmatrix} 0.80 & 1.20 \\ 0.86 & 1.43 \\ 0.93 & 1.39 \end{pmatrix}$$

All three words at once; the last rows are the "sat" numbers of the table.

Computed directly; rounded to two decimals.

Self-Attention in PyTorch

```
import torch, math
import torch.nn.functional as F

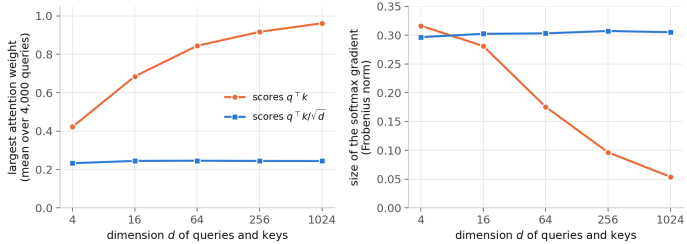
def self_attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv   # (T, d_k)
    S = Q @ K.T / math.sqrt(K.shape[-1])
    A = S.softmax(dim=-1)    # rows sum to 1
    return A @ V, A

X  = torch.tensor([[1., 0], [0, 1], [1, 1]])
Wq = torch.tensor([[1., 0], [1, 1]])
Wk = torch.tensor([[1., 1], [0, 1]])
Wv = torch.tensor([[1., 0], [0, 2]])
Z, A = self_attention(X, Wq, Wk, Wv)

# the built-in version that real models call
Z2 = F.scaled_dot_product_attention(
    X @ Wq, X @ Wk, X @ Wv)
torch.allclose(Z, Z2)   # True
```

- Scores, softmax and weighted sum are one line each.
- Run on the three-word example, A comes out as $(0.40, 0.20, 0.40)$, $(0.28, 0.14, 0.58)$, $(0.31, 0.07, 0.62)$.
- No loop over positions: two matrix products and a softmax process every token at once.
- `scaled_dot_product_attention` gives the same Z, and also handles batches and heads.

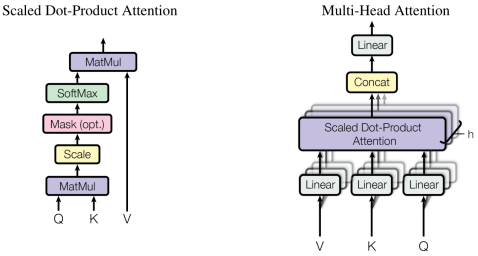
Why Divide by $\sqrt{d_k}$, Measured



Random queries and 16 random keys with independent standard normal entries; 4,000 trials per dimension. Computed for this course.

- **The problem:** with entries of mean 0 and variance 1, $q^\top k$ has variance d_k (Vaswani et al., 2017, footnote 4); its standard deviation grew from 2 at $d = 4$ to 32 at $d = 1024$. Unscaled, the softmax then puts on average 96% of the weight on one key at $d = 1024$, and its gradient shrinks to under a fifth.
- **The fix:** divide the scores by $\sqrt{d_k}$, which brings their standard deviation back to 1. Weights and gradient then stay flat at every size.

Multi-Head Attention



Vaswani et al., arXiv:1706.03762v7, Figure 2.

- **The problem:** one softmax gives one weighted average per word, so one attention, called a **head**, can follow only one kind of relation at a time: what "it" refers to, or which word came just before.
- **The fix:** run h attentions in parallel, each with its own projections, and combine them:

$$\text{head}_i = \text{Attention}\left(XW_Q^{(i)}, XW_K^{(i)}, XW_V^{(i)}\right)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

- Each head works in d_{model}/h dimensions, so h heads cost the same as one full-size head: $4\, d_{\text{model}}^2$ weights per layer (W_Q, W_K, W_V, W_O) whatever h is. The original model: $d_{\text{model}} = 512$, 8 heads of 64.

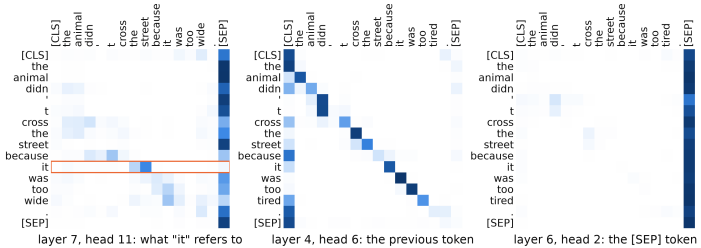
Multi-Head Attention, Shape by Shape

step	operation	shape
input	10 tokens	10×768
project	$Q = XW_Q$ (same for K, V)	10×768
split heads	reshape into 12 heads	$12 \times 10 \times 64$
scores	$QK^\top / \sqrt{64}$ per head	$12 \times 10 \times 10$
weights	softmax over the last axis	$12 \times 10 \times 10$
mix	weights $\times$ V per head	$12 \times 10 \times 64$
concatenate	join the 12 heads	10×768
output	$\times W_O$	10×768

Sizes of DistilBERT, a trained English encoder: $d_{\text{model}} = 768$, $h = 12$, $d_k = 64$; a 10-token input, batch dimension left out.

- The input and output of the layer have the same shape, which is what lets a residual connection add them.
- The 12 heads are 12 independent 10×10 weight matrices computed in one batched product.
- W_O (768×768) mixes what the heads found back into one vector per token.
- Weights per layer: $4 \times 768^2 = 2{,}359{,}296$ plus biases.

Real Heads Do Different Jobs, Measured



Three of the 144 heads of `bert-base-uncased` (12 layers $\times$ 12 heads); rows attend to columns. `[CLS]` and `[SEP]` are the start and end markers BERT adds to every input. Computed for this course.

- **Left:** "it" follows the meaning. When it was too wide, 0.59 of its weight goes to "street"; in "... because it was too tired", 0.28 goes to "animal" and 0.07 to "street". We searched all 144 heads for the largest change; most heads show no such pattern.
- **Middle:** 58% of each token's weight on the token before it, a learned bigram view. **Right:** 92% on `[SEP]`: a head with nothing to add parks its weight there. Clark et al. (2019) also found heads for direct objects of verbs and for coreference.

The Cost: One Score per Pair of Tokens

- $QK^\top$ has n^2 entries for n tokens, in every head of every layer. Doubling the text quadruples the scores.
- It is one reason every model has a maximum input length, its **context window**, counted in tokens.
- The tokenizer decides n. With GPT-2's English-centred tokenizer, Arabic needed 3.7 times the tokens of the same English text, so **13.7 times** the attention scores, and a window fits only about a quarter as much Arabic text.
- With Qwen3's multilingual tokenizer the factor is 1.4 in tokens and 2.0 in scores.

tokens n	scores n^2	memory
512	262,144	0.5 MiB
4,096	16.8 million	32 MiB
32,768	1.07 billion	2 GiB

One head, one layer, 16-bit numbers; computed directly.

tokenizer	tokens, Ar / En	scores
GPT-2	3.71	13.7
BERT (English)	2.87	8.2
Qwen3	1.41	2.0
XLM-R	1.04	1.1

Token ratios measured for this course on 913 English-Arabic sentence pairs from OPUS-100 (Zhang et al., 2020); scores: the ratio squared.

Self-Attention Ignores Word Order, Measured

- **The problem:** scores compare vectors, and the weighted sum does not care where a vector came from. Shuffle the inputs and each word gets exactly the same output, in the shuffled order.
- Then "dog bites man" and "man bites dog" give "dog" the same vector, and so does every reordering.
- **The fix:** add a **position code** p_t to each input before the first layer: $x_t + p_t$. Now the same word at a different position is a different input.
- A recurrent network never needed this: the order in which it reads the words is the position.

The permutation test

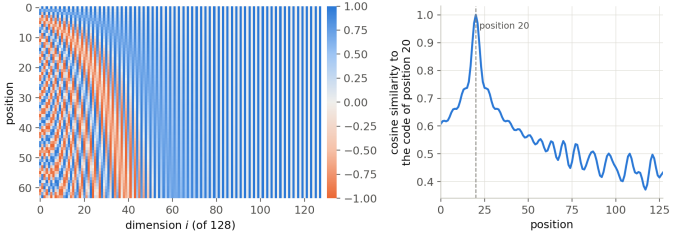
One self-attention layer (2 heads, width 16, random weights), 5 random input vectors, and the order 4, 1, 5, 2, 3:

largest output change	
without positions	6×10^{-8}
with sinusoidal positions	0.26
typical output size	0.15

Without positions, the only difference is floating-point rounding.

Computed for this course with `torch.nn.MultiheadAttention`.

Sinusoidal Position Encodings



Left: the codes for 64 positions in 128 dimensions. Right: cosine similarity of every position's code to the code of position 20. Computed for this course.

$$p_{t,\,2i} = \sin\left(t/10000^{2i/d}\right)$$

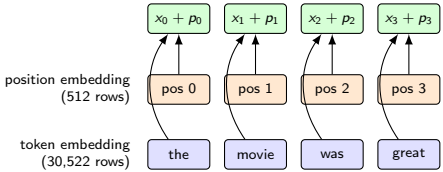
$$p_{t,\,2i+1} = \cos\left(t/10000^{2i/d}\right)$$

t: position; i: dimension pair; d: model width. Wavelengths run from 2π to $10000 \cdot 2\pi$.

- Early dimensions change fast, late ones slowly, like the hands of a clock; together they give every position a unique code.
- Nearby positions get similar codes. Vaswani et al.: for any offset k, p_{t+k} "can be represented as a linear function of" p_t.

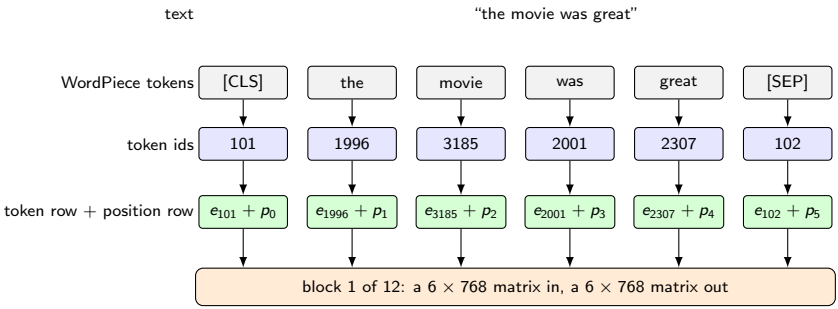
Learned Position Embeddings

- The alternative is a second embedding table, one learned row per position, trained with the rest of the model.
- Vaswani et al. tried both and "found that the two versions produced nearly identical results"; they kept the sinusoids because they might extend to longer sequences than seen in training.
- BERT and DistilBERT learn 512 position vectors of size 768: $512 \times 768 = 393{,}216$ weights. Position 513 has no row, so the input length is capped at 512 tokens.
- Both kinds encode **absolute** position, though what usually matters is **relative**: "the word two to the left". Most current large language models, Qwen3 among them, use **rotary embeddings** (RoPE; Su et al., 2021): queries and keys are rotated by an angle that grows with position, so a score depends on how far apart two tokens are.



Input to the first block of BERT: the sum of a token row and a position row (BERT also adds a segment row).

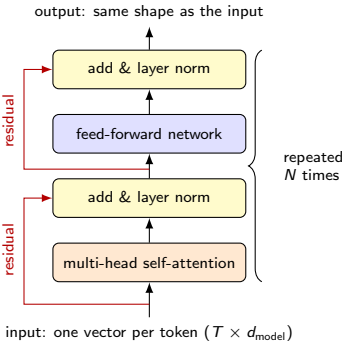
From Text to the First Block



Tokens and ids from the `bert-base-uncased` tokenizer; computed for this course.

- A transformer is a stack of identical **blocks**, each taking one vector per token and returning one vector per token. To make the first input, each id selects one 768-number row of the token embedding table, and the position row is added.
- `[CLS]` (id 101) and `[SEP]` (id 102) are added by the tokenizer itself. Every later block takes the previous block's 6×768 output.

Inside One Encoder Block



- **The problem:** self-attention mixes information **across** tokens, but its output is only a weighted average of linear projections: little is computed within each token.
- **The fix, a block:** self-attention, then a feed-forward network that transforms each token **on its own**. Each sublayer is wrapped as $\text{LayerNorm}(x + \text{Sublayer}(x))$ in the original model.
- Input and output have the same shape, so blocks stack: $N = 6$ in the original model, 6 in DistilBERT, 12 in BERT-base, 28 in Qwen3-0.6B. Each block has its own weights.

Residual Connections and Layer Normalization

Residual connection

- **The problem:** a stack of 6 to 28 blocks is a deep network, and a gradient that passes through every block can shrink, as through the steps of an RNN.
- **The fix:** each sublayer adds a correction to its input instead of replacing it: $x + f(x)$.
- The gradient of $x + f(x)$ with respect to x contains the identity, so a path with no multiplications runs from the loss back to the first block, the same reason the LSTM cell state helped.

Layer normalization (Ba, Kiros and Hinton, 2016)

- **The problem:** as each block adds to its input, the scale of a token's vector drifts from block to block, which makes training unstable.
- **The fix:** for each token separately, rescale its d features to mean 0 and variance 1, then apply a learned scale γ and shift β:

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta, \quad \mu = \tfrac{1}{d} \textstyle\sum_j x_j$$

  with σ the standard deviation of the same d numbers.
- Batch normalization averages over the batch; layer normalization uses only the token's own features, so it works for any batch size and any sequence length.

Ba, Kiros and Hinton, "Layer Normalization," 2016, arXiv:1607.06450v1.

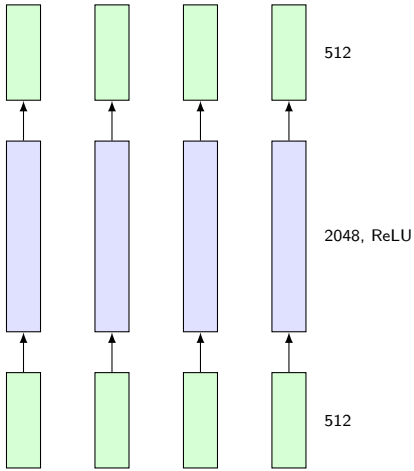
The Feed-Forward Network

- Applied "to each position separately and identically":

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\, W_2 + b_2$$

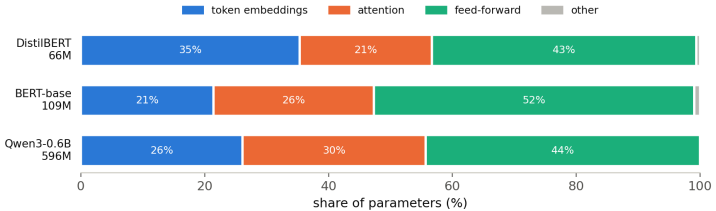
  W_1 widens the vector from d_{model} to d_{ff}, ReLU, W_2 narrows it back.

- Original model: $d_{\text{model}} = 512$, $d_{\text{ff}} = 2048$, so $2 \cdot 512 \cdot 2048 \approx 2.1\text{M}$ weights against $4 \cdot 512^2 \approx 1.05\text{M}$ for attention: two thirds of each block.
- Attention decides which tokens' information to combine; the feed-forward network computes new features from what each token now holds.



Vaswani et al., arXiv:1706.03762v7, Equation 2.

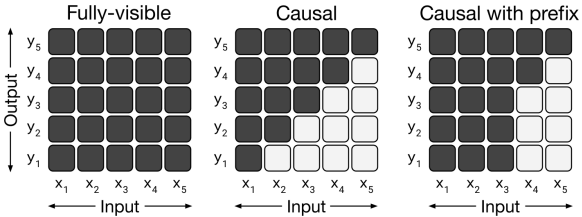
Where the Parameters Are, Counted



Counted for this course from the model configurations (shapes only). "Other": position embeddings, layer norms and BERT's pooler.

- In every model the feed-forward networks hold the largest share: two thirds of the block weights in BERT and DistilBERT, 60% in Qwen3-0.6B.
- Embeddings are a large share of small models: Qwen3-0.6B's embedding matrix (151,936 rows of 1,024) takes 156M of its 596M weights. The same matrix is reused as the output layer (tied embeddings), so it is counted once.
- DistilBERT (Sanh et al., 2019) halves BERT-base's 12 blocks to 6; the embeddings stay the same size, so the total falls from 109M to 66M.

Causal Masking



Dark cell: output y_i may attend to input x_j. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," JMLR 2020, arXiv:1910.10683v4, Figure 3.

- **The problem:** a decoder generates text: it predicts token $t+1$ from tokens $1 \dots t$. Trained on whole texts at once, self-attention would let position t simply read token $t+1$: the answer.
- **The fix, a causal mask:** add $-\infty$ to every score above the diagonal before the softmax. Those weights become $e^{-\infty} = 0$, and each row renormalises over what is allowed.
- **Fully visible**: the encoder. **Causal**: the decoder. **Causal with prefix**: a prompt read fully, then generation.

Causal Masking by Hand

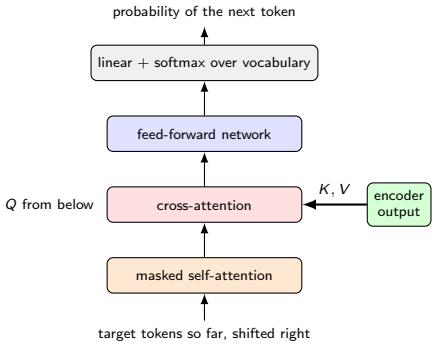
- Three tokens, “the cat sat”, with scaled scores and values computed as in self-attention. M holds 0 on and below the diagonal and $-\infty$ above it.
- “the” may only attend to itself; “cat” to “the” and itself; “sat” to everything, so its row is unchanged.
- Without the mask, the row of “the” was $(0.40, 0.20, 0.40)$. Now its output holds nothing about “cat” or “sat”: predicting the next word from it is no longer cheating.

$$\frac{QK^\top}{\sqrt{2}} + M = \begin{pmatrix} 0.71 & -\infty & -\infty \\ 1.41 & 0.71 & -\infty \\ 2.12 & 0.71 & 2.83 \end{pmatrix}$$

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0.67 & 0.33 & 0 \\ 0.31 & 0.07 & 0.62 \end{pmatrix}, \quad Z = AV = \begin{pmatrix} 1.00 & 0 \\ 0.67 & 0.66 \\ 0.93 & 1.39 \end{pmatrix}$$

```
mask = torch.ones(3, 3).triu(1).bool()
S = S.masked_fill(mask, float("-inf"))
A = S.softmax(dim=-1)          # the A above
Z = F.scaled_dot_product_attention(
    Q, K, V, is_causal=True)  # same Z, built in
```

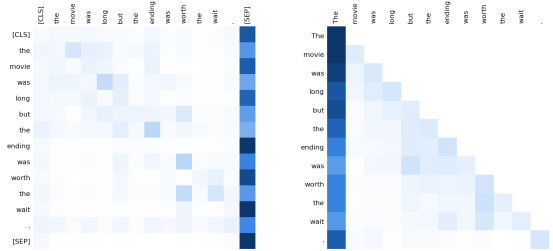
The Decoder Block Has Three Sublayers



Add & layer norm after each sublayer omitted for space.

- To write text, a block needs two things the encoder block lacks: the causal mask, and a way to read the source.
- **Masked self-attention:** self-attention with the causal mask, over the target tokens written so far.
- **Cross-attention:** queries from the decoder, keys and values from the encoder's final output, which every decoder block reads.
- **Feed-forward network**, as in the encoder.
- A linear layer turns the last vector into one score per vocabulary token. Many models reuse the input embedding matrix for it, as Qwen3-0.6B does.
- A decoder-only model such as GPT or Qwen has no encoder and drops the cross-attention.

An Encoder and a Decoder on One Sentence, Measured



Attention averaged over the heads of one middle layer; rows attend to columns. Computed for this course.

- Qwen3's map is exactly zero above the diagonal (the largest such weight in any layer or head was 0.0): the causal mask at work. BERT's is filled on both sides.
- Both park much of their weight on one token, BERT on `[SEP]` and Qwen3 on the first token: a place for the weight of a head with nothing better to attend to.

One Forward Pass Trains Every Position

Training

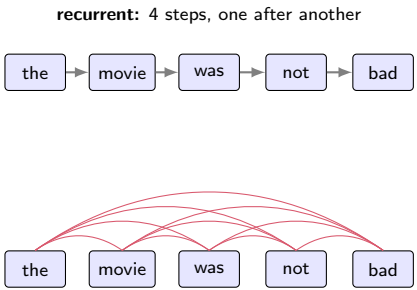
- Feed the whole target sequence at once (teacher forcing). Thanks to the mask, position t sees only tokens $\leq t$ and predicts token $t+1$.
- A 1,000-token text gives 1,000 next-token predictions and losses from a single forward pass, all computed in parallel.
- A recurrent decoder needs 1,000 sequential steps for the same losses.

Generation

- No future text exists, so generation is still one token per forward pass: predict, append, run again.
- Parallel training and sequential generation: the reason training long texts is cheap and generating them is slow.

Why Transformers Won: No Chain of Steps

- **Parallel training.** A recurrent layer must finish word t before it can start word $t+1$: a 1,000-word text is 1,000 steps one after another. Self-attention handles every position in one batch of matrix products, which is what GPUs are built for.
- **Short paths.** Word 1 reaches word n through n recurrent steps, each a chance for the gradient to shrink; in self-attention, through one layer.
- **Scale.** Because training uses every position at once, more GPUs and more text turn into bigger models; extra GPUs cannot shorten a recurrent chain.
- **The price:** n^2 scores per layer. Cheap for sentences, expensive for long documents.



Better Translation for Less Training

model	BLEU EN-DE	BLEU EN-FR	training cost (FLOPs) EN-DE	training cost (FLOPs) EN-FR
GNMT + RL	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
Transformer (base)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

Single models; English to German (EN-DE) and English to French (EN-FR) test sets of the 2014 Workshop on Machine Translation; FLOPs: floating-point operations. Vaswani et al., arXiv:1706.03762v7, Table 2 (selected rows).

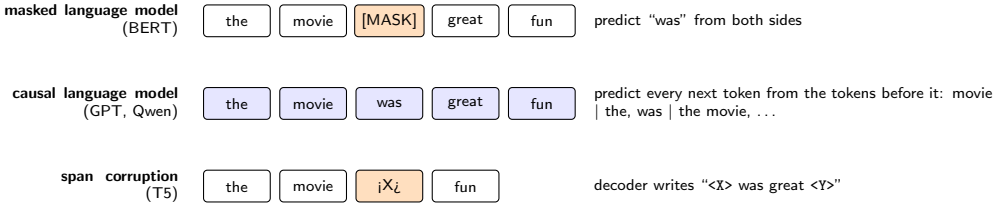
- GNMT (Wu et al., 2016) is Google's LSTM system with 8 encoder and 8 decoder layers; ConvS2S (Gehring et al., 2017) is built entirely from convolutions.
- The base transformer beat both on English to German with about a seventh of GNMT's training compute.
- Training time: 12 hours for the base model and 3.5 days for the big one, on 8 GPUs.

Pretraining: Each Family Learns From Raw Text

- **The problem:** labelled examples are few (IMDb has 50,000 labelled reviews), raw text is nearly unlimited. **The fix, pretraining:** first train on raw text with a task whose answers come from the text itself; then **fine-tune** on the labelled task.
- The task must fit what the model sees. An encoder sees both sides of every word, so asking it for the next word would be no test: it can read it. The word must be hidden first.

	encoder-only	**decoder-only**	**encoder-decoder**
each token sees	both sides	only earlier tokens	the whole input; earlier output tokens
pretraining task	predict masked words	predict the next token	rebuild corrupted spans
the answer is	the hidden words	the next token	the removed spans
examples	BERT, AraBERT, multilingual E5	GPT, Qwen3	T5, BART, mBART

One Sentence, Three Pretraining Objectives



- A masked model sees both sides of the gap, which suits understanding tasks, but it cannot generate left to right.
- A causal model gets a training signal from every token and can generate, but each token’s vector is built only from the words before it.
- Span corruption trains an encoder to read a damaged text and a decoder to write what is missing.

Encoders: Masked Language Modelling

- BERT (Devlin et al., 2019) chooses 15% of the token positions for prediction and replaces the token with `[MASK]` 80% of the time, a random token 10%, and leaves it unchanged 10%, so the model cannot rely on seeing `[MASK]`.
- Trained on BooksCorpus (800M words) and English Wikipedia (2,500M words). BERT-base: 12 blocks, width 768, 12 heads, 110M parameters.
- A special `[CLS]` token at the start of every input gives one vector for the whole text, used for classification.

What `bert-base-uncased` predicts

"I `[MASK]` this movie, the acting was . . ."		
word	**terrible**	**wonderful**
loved	0.386	0.638
hated	0.275	0.002
liked	0.087	0.085

Probabilities for the masked position. "Paris is the `[MASK]` of France." gives "capital" 0.997. Computed for this course.

- The words after the gap change the prediction: "hated" goes from 0.002 to 0.275. A left-to-right model could not use them.

AraBERT: An Encoder Pretrained on Arabic

- Same architecture as BERT-base, pretrained on Arabic text only. The first version (Antoun et al., 2020) used 70 million sentences, about 24GB, mostly news.
- AraBERTv0.2-base: 136M parameters, 200 million sentences, 77GB, 8.6 billion words. Unlike v1 and v2, it reads raw text, without first splitting off prefixes and suffixes with the Farasa segmenter.
- Its 64,000-token vocabulary is spent on Arabic: on 913 English-Arabic sentence pairs from OPUS-100, it used 1.3 tokens per Arabic word, against 5.7 for GPT-2's tokenizer (measured for this course).

Antoun, Baly and Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," OSACT 2020, arXiv:2003.00104v4; model card of aubmindlab/bert-base-arabertv02.

What `bert-base-arabertv02` predicts

. عاصمة المملكة العربية السعودية هي [MASK]

(The capital of Saudi Arabia is [MASK].)

الرياض 0.997, جدة 0.002

. ذهبت إلى [MASK] لشراء الخبز

(I went to [MASK] to buy bread.)

السوق 0.27, المتجر 0.10, مخبز 0.09, الفرن 0.05

Top predictions with probabilities: the market, the store, a bakery, the bakery (oven). Computed for this course.

Decoders: Predict the Next Token

- Objective over a text $x_1 \dots x_T$, with the causal mask:

$$L = -\sum_{t=1}^{T} \log p\left(x_t \mid x_1, \dots, x_{t-1}\right)$$

  the same objective as an n-gram or RNN language model, with a far longer and richer context.
- GPT (Radford et al., 2018): a 12-block decoder pretrained on books, then fine-tuned; it improved the state of the art "in 9 out of the 12 tasks studied".
- Qwen3-0.6B: 28 blocks, width 1,024, 16 attention heads, rotary positions, a vocabulary of 151,669 byte-level BPE tokens.

What Qwen3-0.6B continues, given raw text

"The capital of Saudi Arabia is" $\rightarrow$	
Riyadh	0.620
the	0.056
located	0.045

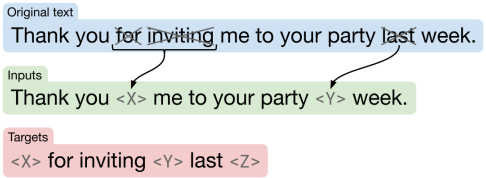
Continued by always taking the most probable token:

"Riyadh. The capital of the United States is Washington, D"

- A model trained only to predict the next token continues the text pattern: a list of capitals, not an answer and a stop.

Next-token probabilities and a 12-token continuation; computed for this course.

Encoder-Decoders: Span Corruption and Denoising



Raffel et al., arXiv:1910.10683v4, Figure 2.

`t5-large` on the same input writes: `<extra_id_0>` for inviting `<extra_id_1>` this `<extra_id_2>` ... and then repeats spans. Computed for this course.

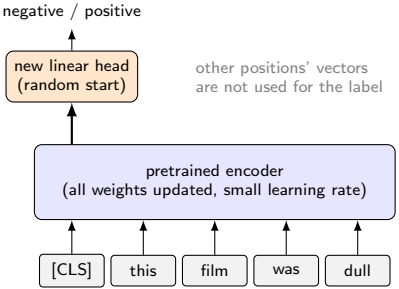
- T5 drops 15% of the tokens; each run of dropped tokens becomes one sentinel (`<X>`, `<Y>`). The encoder reads the damaged text, the decoder writes only the missing spans.
- BART (Lewis et al., 2020) corrupts more freely, for example by text infilling and sentence permutation, and the decoder rewrites the whole original.
- mBART (Liu et al., 2020) applies the BART objective to 25 languages of web text, Arabic among them, before fine-tuning for translation.

Which Family for Which Job

task	family	why
sentiment, topic, spam	encoder	one label from the whole text; both directions help
named entities, tagging	encoder	one label per token, with context on both sides
embeddings for search	encoder	one vector per text, fast to compare
translation, summarisation	encoder-decoder or decoder	the output is new text conditioned on an input
open generation, chat	decoder	generates left to right; one model serves many tasks through prompts

- Size matters as much as family: a 66M-parameter DistilBERT classifier runs on a CPU; a decoder large enough to classify well from a prompt usually does not.
- Large decoders such as Qwen3 can do the encoder jobs too, through instructions, at a higher cost per input.

Pretrain Once, Fine-Tune for Each Task



- **The problem:** a pretrained encoder outputs one vector per token, and the task needs a label. **The fix:** add one small layer on top of the `[CLS]` vector, then train the whole network on the labelled task data with cross-entropy.
- BERT's recommended ranges: learning rate 5×10^{-5}, 3×10^{-5} or 2×10^{-5}; batch size 16 or 32; 2 to 4 epochs. Small rates keep what pretraining learned.
- The expensive part, pretraining, is paid once by someone else; fine-tuning a small model takes minutes to hours on one GPU.
- Howard and Ruder (2018): "with only 100 labeled examples, it matches the performance of training from scratch on 100× more data".

Fine-Tuning in Code: What Is New and What Is Reused

```
import torch
from transformers import (AutoTokenizer,
    AutoModelForSequenceClassification)

name = "distilbert/distilbert-base-uncased"
tok = AutoTokenizer.from_pretrained(name)
model = AutoModelForSequenceClassification\
    .from_pretrained(name, num_labels=2)

batch = tok(["this film was dull"],
            return_tensors="pt")
out = model(**batch, labels=torch.tensor([0]))
out.loss.backward()   # gradients for all layers
```

- The tokenizer must be the one the model was pretrained with: the embedding table only knows its ids.
- Loading reports the classifier weights as newly initialised: they are the only part that starts from random.
- `labels` makes the model return the cross-entropy loss; an optimiser step then updates every weight.

Fine-Tuning on IMDb with the Trainer

```
from datasets import load_dataset
from transformers import Trainer, TrainingArguments

imdb = load_dataset("stanfordnlp/imdb")
split = imdb["train"].train_test_split(
    test_size=0.1, seed=42)
enc = lambda b: tok(b["text"], truncation=True,
                    max_length=256)
train = split["train"].map(enc, batched=True)
val = split["test"].map(enc, batched=True)

args = TrainingArguments(
    output_dir="distilbert-imdb",
    learning_rate=2e-5, num_train_epochs=2,
    per_device_train_batch_size=32,
    eval_strategy="epoch",
    fp16=torch.cuda.is_available())
trainer = Trainer(model=model, args=args,
    train_dataset=train, eval_dataset=val,
    processing_class=tok)
trainer.train()
```

- **The problem:** one training step is not fine-tuning; 22,500 reviews need batches, padding, an optimiser with a learning-rate schedule, and evaluation. **The fix:** the Trainer runs that loop.
- `model` and `tok` are DistilBERT and its tokenizer, loaded with `from_pretrained`.
- Reviews are cut at 256 tokens. Passing the tokenizer lets the Trainer pad each batch only to its longest review.
- **Measured:** 91.3% on all 25,000 test reviews after 2 epochs, with this recipe plus a warmup, a cosine learning-rate schedule and weight decay 0.01.

Reviews Are Longer Than the Window

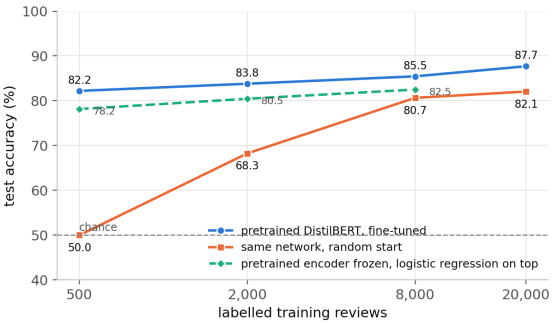
IMDb test reviews in DistilBERT tokens	
median length	230
90th percentile	598
longest	3,157
longer than 128 tokens	88.7%
longer than 256 tokens	43.4%
longer than 512 tokens	14.1%

All 25,000 test reviews, measured for this course.

- BERT-style models stop at 512 tokens; shorter limits are often chosen to train faster, since cost grows with length (and attention with its square).
- Everything after the limit is cut. At 128 tokens, nearly 9 reviews in 10 lose their ending, which is often where the verdict is.
- Sun et al. (2019) fine-tuned BERT on IMDb with 510 tokens: keeping the first 128 and the last 382 (5.42% test error) did slightly better than keeping only the first 510 (5.63%).
- Other options: split a long text into chunks and combine their predictions, or pick a model with a longer window.

Sun, Qiu, Xu and Huang, "How to Fine-Tune BERT for Text Classification?", 2019, arXiv:1905.05583v3, Table 2.

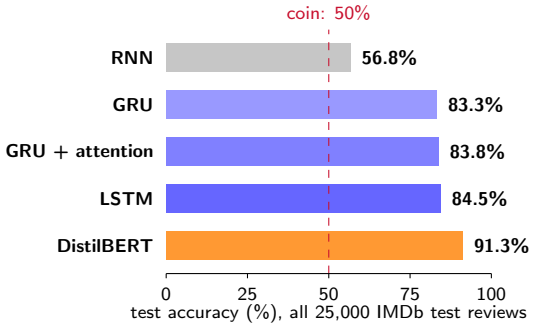
Pretrained Versus From Scratch, Measured



DistilBERT on IMDb reviews (Maas et al., 2011), first 128 tokens, 2 epochs, learning rate 5×10^{-5}; accuracy on the same 2,000 test reviews. Frozen: mean of the last layer's vectors fed to logistic regression. Computed for this course on CPU.

- Same architecture, same recipe, same data; only the starting weights differ.
- From random weights, 500 reviews teach nothing (50%, chance). Pretrained, the same 500 give 82.2%.
- The random start needs 20,000 reviews to reach 82.1%, what the pretrained model had with 500: 40 times the labelled data.
- Even frozen, with only a linear classifier trained, the pretrained encoder beats training from scratch up to 8,000 reviews; fine-tuning all its weights adds 3 to 4 points.

Five Models on the Same 25,000 Reviews



- The four recurrent models learned everything from about 22,500 labelled reviews. DistilBERT was pretrained on billions of words, then fine-tuned on the same reviews.
- Gates were worth 27 points; attention on top of a GRU, half a point; pretraining, 7 more points over the best model trained from scratch.
- The recipes differ in details (words or subword tokens, 200 words or 256 tokens, epochs), so read the gaps as sizes, not exact values.

Each model trained once for this course on CPU and tested once, after training, on the full IMDb test split (Maas et al., 2011).

Recurrent models: 64- or 128-number word embeddings learned from scratch, state of 128.

When Training From Scratch Is Worth It

Train from scratch only if

- no pretrained model covers your language, script or kind of data;
- you have billions of tokens of raw text and the compute to use them;
- you need a tokenizer or architecture of your own, for example to stop paying a token premium on Arabic, or to meet a hardware limit.

Otherwise fine-tune, and choose the starting point

- A model pretrained on your language (AraBERT for Arabic) or a multilingual one.
- A smaller model if latency matters: DistilBERT is 40% smaller and 60% faster than BERT while "retaining 97% of its language understanding capabilities" (Sanh et al., 2019).

The middle path: continue pretraining

- Keep training the pretrained model with its own objective on unlabelled text from your domain, then fine-tune on the labelled task.
- Gururangan et al. (2020): across four domains and eight classification tasks, "a second phase of pretraining in-domain . . . leads to performance gains, under both high- and low-resource settings".
- It suits domains with much raw text and few labels: legal, medical, or a company's own documents.

Sanh et al., "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," 2019, arXiv:1910.01108v4;

Gururangan et al., "Don't Stop Pretraining: Adapt Language Models to Domains and Tasks," ACL 2020, arXiv:2004.10964v3.

Summary

- **Self-attention rewrites every token from all tokens at once:** $Z = \text{softmax}(QK^\top/\sqrt{d_k})V$, two matrix products and a softmax. Without the $\sqrt{d_k}$ the softmax collapses onto one key (96% of the weight at $d = 1024$).
- **Multi-head attention** runs h narrower heads for the cost of one wide head ($4d^2_{\text{model}}$ weights per layer). Real BERT heads resolve "it", look at the previous token, or park on `[SEP]`. The n^2 scores make long inputs, and tokenizers that split Arabic finely, expensive.
- **Attention ignores order**, measured: shuffled inputs give shuffled but identical outputs. Sinusoidal or learned codes add position to the input; BERT's 512 learned positions cap its input length.
- **A block is attention plus a feed-forward network**, each wrapped in a residual connection and layer normalization. The feed-forward networks hold most block weights; embeddings take a quarter to a third of small models.
- **A causal mask** hides future tokens, so one forward pass trains every next-token prediction in parallel, while generation stays one token at a time.
- **Transformers won** because no step waits for the previous word, any two words are one layer apart, and they translated better at a fraction of the training cost.
- **Three families:** encoders with masked-word pretraining (BERT, AraBERT) for classification and search, decoders with next-token pretraining (GPT, Qwen) for generation, encoder-decoders with span corruption (T5, mBART) for text-to-text tasks.
- **Fine-tune, do not train from scratch,** unless no pretrained model fits your language or data. On IMDb, fine-tuned DistilBERT reached 91.3% against 84.5% for the best recurrent model, and from random weights it needed 20,000 labelled reviews to match what the pretrained model reached with 500.

References

1. Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. arXiv:1706.03762v7 (source of Figures 1 and 2, Table 2)
2. Devlin, Chang, Lee and Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” NAACL 2019. arXiv:1810.04805v2
3. Radford, Narasimhan, Salimans and Sutskever, “Improving Language Understanding by Generative Pre-Training,” OpenAI, 2018. cdn.openai.com
4. Qwen Team, “Qwen3 Technical Report,” 2025. arXiv:2505.09388v1; configuration of Qwen3-0.6B, huggingface.co/Qwen/Qwen3-0.6B
5. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” JMLR 2020. arXiv:1910.10683v4 (source of Figures 2 and 3)
6. Tiedemann and Thottingal, “OPUS-MT – Building open translation services for the World,” EAMT 2020 (Marian translation models). aclanthology.org/2020.eamt-1.61
7. CMU 11-785, *Introduction to Deep Learning*, Spring 2024, Lecture 19: Transformers (source of the figures so credited). deeplearning.cs.cmu.edu
8. Clark, Khandelwal, Levy and Manning, “What Does BERT Look At? An Analysis of BERT’s Attention,” BlackboxNLP 2019. arXiv:1906.04341v1

References (cont.)

9. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” 2021. arXiv:2104.09864v5
10. Ba, Kiros and Hinton, “Layer Normalization,” 2016. arXiv:1607.06450v1
11. Sanh, Debut, Chaumond and Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” NeurIPS 2019 Workshop on Energy Efficient Machine Learning and Cognitive Computing. arXiv:1910.01108v4
12. Wu et al., “Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation,” 2016. arXiv:1609.08144v2
13. Gehring, Auli, Grangier, Yarats and Dauphin, “Convolutional Sequence to Sequence Learning,” ICML 2017. arXiv:1705.03122v3
14. Antoun, Baly and Hajj, “AraBERT: Transformer-based Model for Arabic Language Understanding,” OSACT 2020. arXiv:2003.00104v4; model card of aubmindlab/bert-base-arabertv02
15. Zhang et al., “Improving Massively Multilingual Neural Machine Translation and Zero-Shot Translation” (OPUS-100), ACL 2020. arXiv:2004.11867v1

References (cont.)

16. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," ACL 2020. arXiv:1910.13461v1
17. Liu et al., "Multilingual Denoising Pre-training for Neural Machine Translation" (mBART), TACL 2020. arXiv:2001.08210v2
18. Howard and Ruder, "Universal Language Model Fine-tuning for Text Classification," ACL 2018. arXiv:1801.06146v5
19. Sun, Qiu, Xu and Huang, "How to Fine-Tune BERT for Text Classification?", 2019. arXiv:1905.05583v3
20. Maas et al., "Learning Word Vectors for Sentiment Analysis" (the IMDb review dataset), ACL 2011. aclanthology.org/P11-1015
21. Gururangan et al., "Don't Stop Pretraining: Adapt Language Models to Domains and Tasks," ACL 2020. arXiv:2004.10964v3